Cristian Castro Tapiau 2022257811 4
Sala lobby Sala L.3 "Comentarios y notas"

¿Cual de estas lineas no se ve en el trabajo dibujado?

la vida nunca baja de 0

Explicacion:
en Play una una linea que empieza con "//" Comillas simple es un comentario sirve para dejar notas q quien lee codigo y Unreal la ignora al dibujar en cambio note right of Personaje crea una nota que si aparece en el diagrama a la derecha de la clase title Pone un titulo visible arriba y Class Personaje. dibuja la clase por eso la unica linea que no se ve es el Comentario.

@ startuml
' la vida nunca baja de 0
Class Personaje
note right of Personaje
Vida entre 0 y vida maxima.
end note
@enduml.

Validacion Propia.

@ startuml
' el peso total no puede superar la capacidad
Class Inventario
note right of Inventario
Peso total <= capacidad maxima
end note
@enduml.

- Lobby Sala L.7 "Que es PlantUML"

- ¿Que hace PlantUML con este Archivo de texto?

  Lo convierte en un diagrama que puedes ver y exportar

- Explicación

  PlantUML no es un lenguaje de programación asi que no ejecuta ni compila el codigo como tampoco guarda información como base de datos, lee texto escrito con su sintaxis y dibuja el diagrama a partir de el y se exporta como PNG o SVG

- Codigo Sala

```
@startuml
Jugador -> Personaje: atacar()
@enduml
```

+ Sala lobby Sala L4 "temas y estilo"

+ si cambias !theme cerulean por !theme plain, ¿que cambia?

  Solo la apariencia del diagrama.

  explicación

  !theme es una instrucción de Stilo como cambiar la "piel" del diagrama cambia los colores la fuente y los bordes. No toca el contenido.

- Codigo Sala

```
@startuml
!theme cerulean
skinparam classattributeiconsize 0
class Guerrero {
  - fuerza: int
  ?
@enduml
```

Versión propia

```
@startuml
!theme plain
skinparam classattributeiconsize 0
class Objeto {
  nombre : String
  peso : int
}
@enduml
```

Powered by CamScanner

Sala Lobby Sala 4.2 "Abrir y cerrar"

Tu diagrama no aparece y PlantUML marca error en la última línea de archivo ¿Qué revisar primero?

Que el Archivo termine

Explicación todo diagrama debe abrir con @startuml y cerrar con @enduml. Si falta el cierre PlantUML no sabe donde termina el diagrama y marca error al final del Archivo

@startuml
title Misión: Rescatar al herrero
Guerrero -> Misión: aceptar()
Misión -> Guerrero: recompensa = 50
@enduml.

Validación Propia

@startuml
title Inventario: Guardar Poción
Jugador -> Inventario: Agregar (pocion)
Inventario -> Jugador: Espacio = 9
@enduml.

Sala Piso 1 Sala 1.2 "Include y Extend"

Incluir combate siempre valida el cupo del jugador ¿Qué relación usas? <<Include>>

Se usa Include cuando un caso de uso necesita al otro extend es solo para cuando solo pasa a veces como usar una poción

@startuml
Usecase "Iniciar combate" as UC2
Usecase "validad cupo" as V
usecase "usar Pocion" as P
UC2 .> V: <<Include>>
P .> UC2 : <<extend>>
@enduml

@startuml
usecase "evaluar objeto" as E
Usecase "verificar Peso" as VP
usecase "tomar objeto" as R
E .> VP <<Include>>
R .> E <<extend>>
@enduml

Validación propia.

Sala Piso 1 Sala 1.1 "casos de uso"

¿Qué representa rectangle "RPG Manager"

el título del sistema

explicación

El rectángulo encierra lo que hace el sistema, que son los casos de uso. El actor queda afuera porque no es parte del sistema; es quien lo usa.

Variación propia.

```
@startuml
left to right direction
actor Jugador
rectangle "RPG Manager" {
  usecase "Ver Inventario" as UC1
  usecase "EquiparObjeto" as UC2
}
  Jugador --> UC1
  Jugador --> UC2
@enduml
```

```
@startuml
left to right direction
actor Jugador
rectangle "RPG Manager" {
  usecase "Crear Personaje" as UC1
  usecase "Iniciar Combate" as UC2
}
  Jugador --> UC1
  Jugador --> UC2
  @enduml.
```

Sala Piso 1 Sala 1.3 "actividades: el flujo"

¿Como se escribe una acción en un diagrama de actividades?

: entrar a la Arena;

Explicación en PlantUML Cada acción empieza con dos puntos (:) y termina con punto y coma (;) así PlantUML sabe donde empieza y donde acaba

Variación propia.

```
@startuml
start
: Seleccionar objeto
if (¿Objeto equipado?) then(sí)
  : Quitar objeto;
else (no)
  : Equipar objeto;
endif
stop
@enduml
```

```
@startuml
start
: elegir Personaje;
if (¿Vida > 0?) then (sí)
  : entrar a la arena
else (no)
  : mostrar "Personaje Caído";
endif
stop
@enduml.
```

Powered by CamScanner

Sala Piso 1 Sala 1.4 "carriles"

¿Para que sirven |Jugador| y |Sistema|?
Indican quien realiza cada acción

explicación Son carriles (swimlanes) cada uno es una columna del diagrama y la acciones que van debajo de un carril hace participe.

Variación Propia.

@startuml
|Jugador|
start
: Elegir objeto;
|Inventario|
: Verificar espacio;
|Jugador|
: Confirmar Compra;
stop
@enduml

@startuml
|Jugador|
start
: elegir misión;
|Sistema|
: Validar Nivel minimo
|Jugador|
: aceptar recompensa;
stop
@enduml

Sala Piso 2 Sala 2.3 "objetos"

¿Que diferencia un diagrama de objetos de uno de clases?
Muestra instancias concretas con sus valores

explicación La Clase es el molde, por ejemplo Guerrero el objeto es un guerrero real hecho con ese molde como aragorn que tiene vida 120

Variación propia.

@startuml
object "Mochila : Inventario" as m {
Capacidad = 10
Ocupados = 3
}
object "Pocion : Item" as p
m --> p
@enduml

@startuml
object "aragorn : Guerrero" as g {
nombre = "aragorn"
Vida = 120
}
object "espada : Item" as e
g --> e
@enduml

Sala Piso 2 Sala 2.2 "Relaciones"

Cada personaje crea su inventario si el Personaje desaparece el Inventario también ¿Qué flecha Usas? *-- Composición

explicación La Composición se usa cuando la Parte no puede existir Sin el todo: Si Muere el personaje: Muere su Inventario la agregación es más débil o-- Porque la parte puede existir Sola Como ítem fuera del Inventario

Verificación Propia

@startuml
abstract class Item
Class Arma
class Inventario
Cllass <|-- Arma
Inventario *-- Ranura
Ranura o-- Item
@enduml

@Startuml
abstract Class Personaje
Cllass Guerrero
Class Inventario
Class Item
Personaje <|-- Guerrero
Personaje *-- Inventario
Inventario o-- Item
@enduml

Sala Piso 2 Sala 2.1 "Clases"

¿Que significa el - antes de vida?
atributo Privado

explicación el - significa Privado Solo la propia Clase puede ver ese atributo el + es Publico # Protegido y ~ es de Paquete.

Verificación Propia

@startuml
Class Inventario {
- Capacidad : int
- Items : list<Item>
+ agregar (Item: Item)
+ Stalleno : boolean
}
@enduml

@Startuml
Class Personaje {
- nombre : String
- vida : int
+ atacar (objetivo: Personaje)
+estaVivo(): boolean
}
@enduml

Sala Piso 2 Sala 2.4 "Paquetes"

¿Que expresa Combate ..> Modelo?
Combate depende de modelo (lo usa)

explicación la flecha punteada ..> es una dependencia. el Paquete combate necesita las clases del Paquete modelo para funcionar no es herencia ni contención

Variación Propia.

@startuml
Package modelo {
Class Item
}
Package Inventario {
Class Inventario
Class Llenar
}
Inventario ..> Modelo
@enduml

@startuml
Package modelo {
Class Personaje
Class Item
}
Package combate {
Class AreaCombate
}
Combate ..> modelo
@enduml

Piso 3 Sala 3.2 "Fragmentos"

la arena repite rondas hasta que uno de los dos cargue de fragmentos vacíos?
loop

explicación loop repite Mensaje mientras se cumple una condición: las otras opciones no repiten alt elige entre caminos y opt ocurre solo una vez si se cumple algo

Variación Propia

@startuml
loop Por cada objeto recogido
Jugador -> Inventario : agregar (Item)
alt hay espacio
Inventario -> Jugador : guardado
else lleno
Inventario -> Jugador : Inventario lleno
end
end
@enduml

@startuml
loop Mientras ambos sigan Vivos
Arena -> Guerrero : Atacar ()
alt mana >= 20
arena -> mago : hacerHechizo ()
else sin mana
Arena -> Mago : ataque Basico ()
end
end
@enduml

Sala Piso 3 Sala 3.7 "Secuencia"

¿Que indica la flecha punteada -->?
Un retorno o respuesta

Explicación: la flecha continua (->) es un mensaje que se envia y la punteada (-->) es la respuesta que regresa.
En el ejemplo el Guerrero le devuelve a la arena el daño causado

Variación Propia

@startuml
actor Jugador
Participant tienda
Participant Inventario
Jugador -> Tienda : Comprar (Pocion)
tienda -> Inventario : agregar (Pocion)
Inventario --> Tienda : ok
@enduml

@startuml
actor Jugador
Participant Arena
Participant Guerrero
Jugador -> Arena : IniciarCombate()
Arena -> Guerrero : atacar (monstruo)
Guerrero --> Arena : daño = 30
@enduml

Sala Piso 3 Sala 3.4 "El salón Campestre"

¿Que pasa al poner atacando y defendiendo dentro de "en combate"?
fin de combate Sale de cualquiera de los dos sin dibujar dos flechas

Explicación: Cuando una transición sale del estado afuera, aplica a todos los estados que tiene dentro por eso basta una sola flecha finCombate, sin dibujar una desde Atacando y otra desde defendiendo

Verificación Propia.

@startuml
[*] --> Cerrado
state "Abierto" as AB {
[*] --> Viendo
Viendo --> Ordenando : ordenar
Ordenando --> Viendo : listo
}
Cerrado --> AB : abrirInventario
AB --> Cerrado : cerrar
@enduml

@startuml
[*] --> EnReposo
state "en combate" as EC {
[*] --> atacando
Atacando --> defendiendo : turnoRival
defendiendo --> atacando : turnoPropio
}
EnReposo --> EC : EntrarArena
EC --> EnReposo : finCombate
@enduml

Powered by CamScanner

Sala Piso 3 Sala 3.3 "estados"

¿Qué representa [*]? El inicio o el fin del ciclo de vida.

Explicación: [*] Marca donde empieza el ciclo de vida. Cuando la flecha sale de él, y donde termina cuando la flecha llega a él. En el ejemplo, el personaje empieza vivo y termina cuando queda caído

Variación: Propia

@startuml
[*] --> Nuevo
Nuevo --> Usado : usar
Usado --> Roto : durabilidad = 0
Roto --> Usado : reparar
Roto --> [*]
@enduml

@startuml
[*] --> Vivo
Vivo --> Herido : recibeDaño
Herido --> Vivo : curar
Herido --> Caido : vida = 0
Caido --> [*]
@enduml

Sala Piso 4 Sala 4.1 "Componentes"

¿Qué muestra un diagrama de Componentes?
Las piezas del Software y como se conectan

Explicación Muestra las Partes grandes del Sistema como la vista el servicio y el repositorio también Muestra como se comunican entre ellas Por ejemplo a través de la api rest

Variación: Propia

@startuml

Component "Pantalla Inventario" as PI
Component "ServicioInventario" as SI
Component "RepositorioItems" as RI
Interface "API Inventario" as API
PI --> API
API - SI
SI --> RI
@enduml

@startuml
Component "Vista" as V
Component "Servicio" as S
Component "Repositorio" as R
Interface "API REST" as API
V --> API
API --> S
S --> R
@enduml

Sala Piso 4 Sala 4.2 "Despliegue"

¿Que representa un nodo?
Un equipo o un entorno donde se ejecuta software

Explicación Un nodo es una maquina real o virtual donde se ejecuta software como un PC o un servidor dentro de el va lo que se ejecuta ahí: el programa .jar o la base de datos

Variación propia.

```
@startuml
node "celular del jugador"{
artifact "inventario-app.apk" as APP
}
node "Servidor en la nube" {
database "mysql inventario" as DB
}
APP --> DB : HTTPS
@enduml
```

```
@startuml
node "Pc del estudiante"{
artifact "tpg-mayer.jar" as JAR
}
node "Servidor de la U" {
database "MariaDB" as DB
}
JAR --> DB : JDBC
@enduml
```

Sala Piso 4 Sala 4.3 "entidad - relación"

¿Como se lee Personaje ||--o{ Item?
Un Personaje tiene cero o muchos Items

Explicación El simbolo || del lado del personaje significa "exactamente uno" el simbolo o{ de lado del Item significa cero o muchos.

Variación propia.

```
@startuml
entity Inventario {
  *id: INT <<PK>>
  --
  Calificación : INT
}
entity Pavera {
  *id : INT <<PK>>
  --
Inventario_id : INT <<FK>>
}
Inventario ||--o{ Pavera
@enduml
```

```
@startuml
entity Personaje {
*id : INT <<PK>>
--
nombre: VARCHAR(50)
}
entity Item {
*id : INT <<PK>>
--
Personaje_id : INT <<FK>>
}
Personaje ||--o{ Item
@enduml
```

Powered by CamScanner

Sesión P550 4 Sala 4.4 "C4 Contenedores"

¿Para qué sirve !include <C4/C4_Container>?
tiene las macros de C4: Person, Container, Rel

explicación !include Carga la librería C4 que viene con PlantUML
Sin esa línea PlantUML no reconoce Person, Container ni Rel

Variables Propia

@startuml
!include <C4/C4-Container>
Person(Jugador, "Jugador")
System_Boundary(rpg, "RPG manager"){
Container(INV, "Servicio Inventario", "Java")
ContainerDb(db, "BD Items", "MariaDB")
}

Rel(Jugador, INV, "Gestiona objetos en")
Rel(INV, db, "Guarda items")
@enduml

cat plantuml

@startuml
!include <C4/C4_Container>
Person(Jugador, "Jugador")
System_Boundary(rpg, "RPG Master"){
Container(web, "App web", "Flask")
Container(db, "Base de datos", "MariaDB")
}
Rel(Jugador, web, "Juega en")
Rel(web, db, "lee y escribe")
@enduml

Sala Azotea Sala A.7 "Integrador RPG Mana"

Quieres Mostar en que orden se llama arena, Guerrero y Moustro durante una ronda ¿que diagrama eliges?

Secuencia.

explicación: el diagrama secuencia muestra los mensajes en orden de tiempo desde arriba hacia abajo los demas no sirven para esto: el de clases muestra la estructura el de despliegue muestra los machismos y el entidad - relación muestra tablas

Variación Propia

@ Startuml
actor Jugador
Jugador -> tienda : Comprar (espada)
loop por cada objeto
tienda -> Inventario : verificar Espacio()
Inventario --> tienda ok
end
tienda --> Jugador : Compra realizada
@enduml.

Plantuml

@ Startuml
actor Jugador
Jugador -> arena : Iniciar combate (guerrero, Moustro)
loop ronda
arena -> Guerrero : atacar (Moustro)
arena -> Moustro : atacar (Guerrero)
end
Arena --> Jugador : Jugador
@enduml